Proof-Bound Enterprise Change: Making Invalid State Transitions Mathematically Inaccessible

Sean Chatman

wasm4pm Project
info@chatmangpt.com

Abstract. Enterprise systems accept change through any surface that can influence state — forms, APIs, database writes, admin panels, and increasingly, AI-agent tool calls — and rely on after-the-fact auditing to reconstruct what happened. We invert this discipline. Extending process mining's admitted-event semantics and boundary predicates into enterprise change governance, we define the *admitted change universe* $A_{\text{enterprise}}$: the set of signed, typed, object-linked, process-admitted, policy-bound, receipt-valid changes. A seven-conjunct acceptance predicate over actor, event, OCEL 2.0 object relations, process state, prior receipt, policy and validator versions, freshness nonce, and a post-quantum signature yields the membership theorem $\text{Accept}(x) \Rightarrow x \in A_{\text{enterprise}}$, a chain-soundness lemma making current state a replayable derivation, and an explicit threat model. Under named assumptions we prove eight inaccessibility propositions: raw database writes, process-state skips, object substitution, backdating, replay, schema-valid fraud, rogue administrator edits, and AI-agent hallucinated actions cannot become accepted enterprise state. We bound the adversary's residual surface by a union over seven characterised break channels — cryptographic, modelling, operational, and engineering — replacing unfalsifiable security claims with an auditable assumption ledger. The framework is implemented in *wasm4pm*, a deterministic WebAssembly process mining platform (60 algorithms), where admitted changes are realised as TrueX envelopes over canonical OCEL 2.0 logs, receipts chain under BLAKE3, the boundary predicate is enforced as thirteen non-overlapping refusal states, and a receipt-backed proof-gate suite guards the gate itself. The contribution is not invincibility but a category shift: from validating the shape of data to proving the admissibility of change.

Keywords: process mining · enterprise change governance · receipt chains · admissibility · post-quantum cryptography · OCEL 2.0 · boundary predicates · AI-agent governance

1 Introduction

Enterprise systems face a structural problem: the surfaces through which change enters the system are too many, too permeable, and too decoupled from the

processes that are supposed to govern them. Consider three incidents, each drawn from a pattern that recurs across industries:

Schema-valid fraud. A payment-operations employee with database access issues `UPDATE invoice SET status = 'paid' WHERE id = 4711`. The row passes every schema constraint — the column accepts the value, the foreign keys resolve, the audit trigger fires and dutifully records the write. Yet no approval event ever occurred. The fraud is invisible to schema validation because the *shape* of the data is correct; only its *process position* is wrong.

Rogue administrator. A platform administrator, asked to "unblock" a stalled onboarding case, sets a user's role to `admin` directly in the identity store. The four-eyes approval workflow that exists precisely for this operation is bypassed, not by exploiting a vulnerability, but by using a privilege the architecture grants: the power to mutate state without emitting the events the process model requires.

AI-agent mutation. An autonomous agent equipped with a write-capable API tool misinterprets an instruction and cancels forty active subscriptions. Every API call is authenticated and authorized; the agent holds a valid service token. What is missing is any binding between the agent's actions and a process law that says *from this state, with this mandate, that transition is admissible.*

In all three cases the failure is the same: **the system accepted a change because it could be parsed and authorized, not because it could be proven to be a lawful step of a declared process.** Current defences respond after the fact: logs are written, anomaly detectors flag deviations [**?**], and audit teams attempt to reconstruct what happened, often months later. Process mining, in van der Aalst's canonical formulation, discovers and verifies process models against event logs recorded after execution [**?**]. The discipline is analytic and retrospective by construction.

The thesis of this paper is that the temporal position of that discipline should be inverted. Instead of mining a log to determine whether a completed process conformed, we require that a *process-valid, cryptographically witnessed event* be produced and admitted *before* the change is permitted to alter enterprise state. Under this regime the unit of enterprise change is no longer the database write or the API call; it is the *admitted change*: a tuple binding actor, event type, object relations, current process state, prior receipt, policy version, validator version, freshness nonce, and a post-quantum signature over all of them. Anything that cannot produce this witness is not interpreted as valid change. It is refused, or it is *residualized* — quarantined as evidence without state authority.

We do not claim such a system is unhackable. We claim something narrower and, we argue, more useful: that under explicitly named cryptographic and implementation assumptions, entire *classes* of adversarial mutation become *mathematically inaccessible to acceptance.* The adversary may still submit anything; what they cannot do — without breaking a named assumption — is make it count.

1.1 Contributions

This paper makes four contributions:

(1) **The Enterprise Change Admissibility Framework** (Section 3). We define the enterprise change universe Ω_{change} and the admitted change set $A_{\text{enterprise}}$ as the subset satisfying a seven-conjunct acceptance predicate. We give the state transition law under which non-admitted changes are identity transitions, prove the membership theorem, and formalise *residualization* as the lossless refusal mode. The framework extends boundary predicates and admitted symbolic events from process mining theory [**?**,**?**] into a governance law for all enterprise state change.

(2) **Eight inaccessibility propositions** (Section 4). We prove that raw database writes, process-state skips, object substitution, backdating, replay, schema-valid fraud, rogue administrator edits, and AI-agent hallucinated actions are each inaccessible to acceptance, identifying for every class the specific predicate conjunct that blocks it. A running procure-to-pay example grounds each proposition in a concrete enterprise scenario.

(3) **A named attack-surface decomposition** (Section 5). We bound the probability of accepting an invalid change by a union over seven named failure channels — cryptographic, algorithmic, modelling, operational, and implementation — each with a characterised mitigation. This replaces unfalsifiable security claims ("zero trust," "secure by design") with an auditable assumption ledger, and connects each channel to the maturity-model literature for operationalisation [**?**,**?**].

(4) **A reference implementation and evaluation** (Section 6). We instantiate the framework in *wasm4pm* [**?**], a process mining platform whose 60 algorithms compile from Rust to deterministic WebAssembly. Admitted changes are realised as *TrueX envelopes* over canonical OCEL 2.0 logs [**?**]; receipts chain under BLAKE3; the boundary predicate is enforced as thirteen non-overlapping refusal states; and signing is post-quantum (ML-DSA [**?**]). We report on the proof-gate test suite, determinism guarantees, and the operational verification protocol.

1.2 Running Example

Throughout the paper we use a procure-to-pay (P2P) process as the running example. An Invoice object progresses through states

$$\mathsf{draft} \rightarrow \mathsf{submitted} \rightarrow \mathsf{approved} \rightarrow \mathsf{paid} \rightarrow \mathsf{archived},$$

driven by event types Submit, Approve, Pay, and Archive. Each event relates the invoice to other objects: the Employee who acts, the PurchaseOrder it references, and the BankTransaction that settles it. The declared process law requires that Approve be performed by an actor distinct from the submitter (four-eyes), that Pay reference an approved invoice and a matching purchase order (three-way match), and that no state be skipped. P2P is chosen because it is the canonical target of enterprise fraud: every attack class in Section 4 has a documented real-world instance in this process.

1.3 Structure

Section 2 reviews process mining, OCEL 2.0, receipt chains, and post-quantum signing. Section 3 develops the formal model, including the threat model, and proves the membership theorem. Section 4 proves the eight inaccessibility propositions. Section 5 gives the attack-surface decomposition and discusses residual risk. Section 6 describes and evaluates the wasm4pm implementation. Section 7 positions the work against eight adjacent research threads. Section 8 concludes with limitations and the category claim.

2 Background

2.1 Process Mining and Object-Centric Event Logs

Process mining operates on event logs to discover process models, check conformance, and enhance existing models [?,?]. The classical event log L is a multiset of traces, each trace a sequence of events over an activity alphabet; an event carries at minimum an activity label, a timestamp, and a case identifier. Conformance checking relates a log to a model either by token replay [?] or by computing optimal alignments [?,?], yielding fitness and precision measures over the four quality dimensions [?]. The control-flow semantics available to a declared model are systematised in the workflow patterns literature [?], and reference implementations of the standard algorithm catalogue are provided by PM4Py [?].

The single-case assumption of classical logs breaks down in enterprise settings where one event touches many entities — a payment references an invoice, a purchase order, and a bank transaction simultaneously. *Object-Centric Event Logs* (OCEL) [?], standardised as OCEL 2.0 [?,?], resolve this: an event is associated with a *set* of typed objects, and the log is formally a structure $(\mathcal{E}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathit{evAttr}, \mathit{objAttr}, \mathit{E2O}, \mathit{O2O})$ comprising events, objects, activities, object types, attribute maps, and the event-to-object and object-to-object relations. Object-centric Petri nets [?] and object-centric directly-follows multigraphs [?] give the discovery-side semantics; OCPQ [?] provides constraint querying over these structures, and declarative object-centric constraints are treated in [?]. Van der Aalst argues that object-centric process intelligence is the prerequisite for trustworthy enterprise AI [?] — a position this paper operationalises by making the object-event relation a signed, first-class component of every admitted change.

In our running P2P example, the Pay event is related to three objects — Invoice #4711, PO #982, BankTx #5530 — and the three-way-match rule is expressible only because the log model carries all three relations on a single event. A classical single-case log would force a choice of case notion and lose exactly the cross-object structure that the fraud checks need.

Boundary predicates. The *boundary predicate* $B : S \times E \to \{0, 1\}$, where S is the set of process states and E the set of event types, encodes whether an event type is admissible from a given state. If $B(s, e) = 0$, the state transition function must satisfy $\mu(s, e) = s$: the event produces no state change. Boundary predicates

appear implicitly throughout the conformance literature as guard conditions on Petri net transitions and as the enabledness relation of workflow nets [?]; recent work on compliance-aware predictive monitoring uses them prospectively [?]. We make the predicate an explicit, versioned, first-class component of the admissibility model: it is evaluated *before* state mutation, its version is bound into the change signature, and its refusals are enumerable (Section 6.4).

2.2 Receipt Chains and the Axiom of Receipt Substance

A *receipt* R_t is a cryptographic record witnessing that the state transition $s_t \xrightarrow{e_t} s_{t+1}$ occurred with specific inputs and outputs. We write $R_t \vdash s_{t+1} = \mu(s_t, e_t)$. Receipts compose into a chain

$$R_0 \to R_1 \to \cdots \to R_n,$$

where each R_t includes the hash $h(R_{t-1})$ of its predecessor, the input hash $h(in_t)$, the output hash $h(out_t)$, and the run identifier. The chain hash function is BLAKE3, which offers 256-bit collision resistance, structural immunity to length-extension attacks, and tree-hash parallelism suitable for streaming over large event logs. Because hash-based constructions rely only on preimage and collision resistance, the chain itself is conservatively post-quantum even before signatures are considered; related type-safe treatments of process evidence appear in [?], and provenance-chain constructions over semantic-web data in [?].

A receipt is *not* a receipt if it carries only hash values without the backing event data. We adopt this as an axiom:

> *A receipt containing only expected hash, observed hash, alignment state, and receipt hash is an unverifiable claim, not evidence.*

The axiom has teeth: it rules out an entire genre of audit theatre in which a producer asserts "hashes matched" without the verifier being able to recompute anything. A valid receipt in our framework must therefore embed, or content-addressably reference, the full OCEL 2.0 logs it witnesses, so that any holder of the receipt can re-canonicalise, re-hash, and re-align. The consequence is formalised as Definition 2.

Proof classes. Receipts are ordered by evidential strength into a proof class hierarchy:

$$\textsc{FixtureProof} \prec \textsc{PredicateProof} \prec \textsc{ChicagoProof} \prec \textsc{BoundaryProof}.$$

A FixtureProof witnesses only that a computation reproduced a stored fixture; a PredicateProof adds machine-checked structural predicates; a ChicagoProof requires end-to-end execution through a real runtime boundary (no mocks at the verified interface); a BoundaryProof adds runtime-observer envelopes and challenge nonces binding the evidence to a specific physical execution. A receipt may not claim a class unless all conditions of that class hold — the refusal state `ProofClassOverclaimed` (Section 6.4) enforces the hierarchy. The hierarchy matters for governance because it gives auditors a vocabulary for *how much* a given receipt proves, in the spirit of capability maturity levels [?].

2.3 Post-Quantum Cryptographic Signing

Classical ECDSA and RSA signatures fall to Shor's algorithm on a cryptographically relevant quantum computer [**?**]. This matters more for enterprise change receipts than for session cryptography: a TLS session key protects data for minutes, but a receipt signed today may need to carry evidential weight in a regulatory proceeding fifteen years from now — squarely inside the harvest-now–decrypt-later window that has driven government migration mandates [**?**]. NIST's 2024 standards provide the replacement primitives: FIPS 204 specifies *ML-DSA* (Module-Lattice-Based Digital Signature Algorithm, derived from CRYSTALS-Dilithium) [**?**], and FIPS 205 specifies *SLH-DSA* (Stateless Hash-Based Digital Signature Algorithm, derived from SPHINCS+) [**?**], whose security rests only on hash-function assumptions and therefore serves as the conservative fallback should lattice assumptions erode.

We model signing abstractly. The authority over a change computes

$$\sigma \;=\; \mathrm{Sign}\big(sk,\; h(a \,\|\, e \,\|\, O \,\|\, s \,\|\, R_{\mathrm{prior}} \,\|\, p \,\|\, v \,\|\, t \,\|\, n)\big),$$

binding the actor a, event type e, object relations O, process state s, prior receipt R_{prior}, policy version p, validator version v, timestamp t, and nonce n under one signature. Verification is $\mathrm{VerifySig}(\sigma, pk, m) \in \{0, 1\}$. The framework is algorithm-agnostic — any EUF-CMA-secure scheme suffices for the proofs of Section 4 — but the implementation default is ML-DSA-65, and the deployment guidance assumes the crypto-agility posture of [**?**]: algorithm identifiers are versioned fields of the receipt, so that a future migration re-signs the chain head rather than rewriting history.

3 Formal Model

We now develop the formal model: the change universe, the acceptance predicate, the state transition law, and the membership theorem, followed by the threat model under which the results of Sections 4 and 5 hold.

3.1 The Change Universe and Admitted Set

Let $\mathcal{A}$ be a set of actors, $\mathcal{E}$ a set of event types, $\mathcal{O}$ a set of typed object-relation sets (each element an OCEL 2.0 event-to-object relation), $\mathcal{S}$ a set of process states, $\mathcal{P}$ a set of policy versions, $\mathcal{V}$ a set of validator versions, $\mathcal{T}$ a set of timestamps, $\mathcal{N}$ a set of nonces, $\mathcal{R}$ a set of receipts, and Σ a set of signatures.

Definition 1 (Enterprise Change Universe). *The* enterprise change universe *is*

$$\Omega_{\mathrm{change}} \;=\; \mathcal{A} \times \mathcal{E} \times \mathcal{O} \times \mathcal{S} \times \mathcal{R} \times \mathcal{P} \times \mathcal{V} \times \mathcal{T} \times \mathcal{N} \times \mathcal{R} \times \Sigma.$$

A candidate change *is a tuple* $x = (a, e, O, s, R_{\mathrm{prior}}, p, v, t, n, R, \sigma)$*: actor* a *proposes event* e *over object relations* O *from process state* s*, citing prior receipt* R_{prior}*, under policy version* p *and validator version* v*, at time* t *with freshness nonce* n*, carrying output receipt* R *and signature* σ*.*

The universe is deliberately maximal: *every* mutation an enterprise can undergo — data edits, status updates, approvals, payments, permission changes, deployments, configuration and policy updates, document mutations, identity changes, model updates, and AI-agent actions — is representable as a candidate change. The framework's discipline consists in what it *accepts* from this universe, not in what can be expressed.

In the running P2P example, the legitimate approval of invoice #4711 is the candidate

$$x_{\mathsf{appr}} = (\mathsf{emp\_207},\ \mathsf{Approve},\ \{\mathsf{Inv\#4711}, \mathsf{PO\#982}\},\ \mathsf{submitted},\ R_{17},\ p_3,\ v_9,\ t,\ n,\ R_{18},\ \sigma),$$

where R_{17} is the receipt of the preceding Submit and R_{18} the receipt this approval emits.

Definition 2 (Valid Receipt). *A receipt R is* valid *for the transition $s \xrightarrow{e} s'$, written* $\mathrm{VerifyReceipt}(R) = 1$*, iff all of the following hold:*

- *(i)* Substance: *R embeds, or references by content address, the complete expected and observed OCEL 2.0 logs of the transition, in canonical serialisation, such that any holder can recompute every hash in R;*
- *(ii)* Chaining: *R carries $h(R_{\mathrm{prior}})$, $h(in)$, and $h(out)$, and $h(R_{\mathrm{prior}})$ equals the hash of the current chain head;*
- *(iii)* Verifier authority: *the alignment state in R was computed and sealed by the verifier; producer-asserted alignment is rejected;*
- *(iv)* Challenge binding: *a nonce minted before execution appears in the challenge manifest, in the raw runner evidence, and inside the observed OCEL events themselves.*

Condition (i) is the formal content of the axiom of receipt substance (Section 2.2); conditions (iii) and (iv) close the two historically exploited shortcuts — self-certification and fixture replay — that Section 6.4 enumerates as refusal states.

Definition 3 (Acceptance Predicate). *For a candidate change $x = (a, e, O, s, R_{\mathrm{prior}}, p, v, t, n, R, \sigma)$ as in Definition 1, define*

$$\begin{aligned}
\mathrm{Accept}(x) \;=\;& \mathrm{VerifySig}\big(\sigma,\ h(a\|e\|O\|s\|R_{\mathrm{prior}}\|p\|v\|t\|n)\big) && \text{(C1)}\\
\wedge\;& \mathrm{VerifyReceipt}(R) && \text{(C2)}\\
\wedge\;& \mathrm{PolicyAllowed}(a, e, p) && \text{(C3)}\\
\wedge\;& \mathrm{ValidatorAllowed}(v) && \text{(C4)}\\
\wedge\;& \mathrm{Fresh}(n) && \text{(C5)}\\
\wedge\;& B(s, e) = 1 && \text{(C6)}\\
\wedge\;& \mathrm{ObjectsValid}(O). && \text{(C7)}
\end{aligned}$$

$\mathrm{Fresh}(n)$ *holds iff n has not been consumed by any previously accepted change.* $\mathrm{PolicyAllowed}(a, e, p)$ *holds iff policy version p is current and grants actor a*

the authority to emit event type e. ValidatorAllowed(v) *holds iff* v *identifies a non-revoked verifier build.* ObjectsValid(O) *holds iff the event-to-object relation* O *is well-typed against the OCEL 2.0 schema and consistent with the object lifecycles recorded in the receipt chain.*

Each conjunct is independently checkable and independently falsifiable; this independence is what powers the decomposition of Section 5. Note also the layering: C1 and C2 are cryptographic, C3–C5 are operational bindings, C6 is the process law, and C7 is the object law.

Definition 4 (Admitted Change Set). $A_{\text{enterprise}} = \{\, x \in \Omega_{\text{change}} \mid \text{Accept}(x) = 1 \,\}$.

Theorem 1 (Acceptance Implies Membership). *For every* $x \in \Omega_{\text{change}}$*:* $\text{Accept}(x) = 1 \Rightarrow x \in A_{\text{enterprise}}$.

Proof. Immediate from Definition 4: $A_{\text{enterprise}}$ is the preimage $\text{Accept}^{-1}(1)$.

Theorem 1 is definitionally true, and we state it anyway, because its force is architectural rather than mathematical: it asserts that *there exists no second path* into enterprise state. In conventional architectures the analogous statement is false — state is reachable through the ORM, through the admin panel, through the migration script, through the integration webhook — and every such path is an acceptance gate that the process model never sees. The theorem is the design obligation that all of those paths either emit admitted changes or lose state authority.

Corollary 1 (Refusal or Residualization). *For every* $x \in \Omega_{\text{change}}$*:* $x \notin A_{\text{enterprise}} \Rightarrow \text{Refuse}(x) \vee \text{Residualize}(x)$.

Refuse(x) rejects the change outright, returning the failing conjunct. Residualize(x) quarantines it: the candidate is recorded, content-addressed, and preserved as investigative evidence, but acquires no state authority. Residualization is the lossless refusal mode — it matters operationally because enterprises cannot simply drop nonconforming inputs (a malformed payment instruction is itself evidence of something), and it matters for the process-mining feedback loop: clusters of similar residuals indicate either an attack campaign or a legitimate behaviour the declared model is missing, in which case the remedy is an explicit, reviewed extension of the process law — never a silent widening of the gate.

Definition 5 (State Transition Law). *The enterprise state transition function* $\mu : \mathcal{S} \times \mathcal{E} \to \mathcal{S}$ *satisfies*

$$\mu(s, e) = \begin{cases} s' & \text{if } B(s, e) = 1 \text{ and } \text{Accept}(x) = 1 \text{ for the change } x \text{ carrying } e, \\ s & \text{otherwise.} \end{cases}$$

Lemma 1 (Chain Soundness). *If every accepted change satisfies Definition 3, then for any reachable state* s_n *there exists a receipt chain* $R_0 \to \cdots \to R_{n-1}$ *such that* $s_n = \mu(\mu(\cdots \mu(s_0, e_0) \cdots), e_{n-1})$ *and each link is independently re-verifiable from the receipt substance alone.*

Proof. By induction on n. The base case is the genesis receipt R_0 witnessing s_0. For the step, the change accepted at position t satisfies C2, so R_t embeds the canonical logs and chains to $h(R_{t-1})$ (Definition 2, i–ii); by the induction hypothesis the prefix is re-verifiable, and condition (i) makes the new link re-verifiable without reference to any state outside the chain.

Lemma 1 is what converts the framework from an access-control discipline into an *evidence* discipline: current state is not a row that happens to exist but the endpoint of a replayable derivation. It is also the precondition for retrospective process mining over admitted history — discovery and conformance algorithms [**?**] run over the chain's embedded OCEL logs with the unusual guarantee that the log is complete and tamper-evident by construction.

3.2 Threat Model

We state the adversary explicitly. The adversary **may**: submit arbitrary candidate changes at any rate; read all public state, receipts, and policies; replay any previously observed message; possess valid credentials for some actors (insider); hold administrative privilege in surrounding infrastructure (database, message bus, file system); and operate AI agents with write-capable tools. The adversary **may not** (assumption ledger, cf. Section 5): forge signatures under EUF-CMA of the deployed scheme; find BLAKE3 preimages or collisions; compromise the validator's signing key; or exploit a defect in the gate implementation. Each "may not" is a named channel in Theorem 2, with its own probability term and mitigation — the framework's security claim is exactly the conjunction of these assumptions, no more.

Two consequences of the threat model deserve emphasis. First, database administrators are *inside* the adversary model: a raw write by a DBA is a candidate change like any other and fails C2 (Proposition 1). The framework therefore does not trust the storage layer — corrupted rows can exist, but they are detectable as divergence between stored state and the receipt-chain derivation of Lemma 1. Second, the verifier is trusted but *versioned and revocable* (C4): a compromised validator build is contained by revoking v, which retroactively identifies every change it admitted.

3.3 The Chesterton Fence as a State Law

Chesterton's principle [**?**] holds that a fence should not be removed until its purpose is understood. The boundary predicate $B(s, e)$ is this principle expressed as a transition law: every enterprise transition is fenced, the fence is versioned and signed, and crossing it requires the full witness of Definition 3. Crucially, the fence does not prevent the *attempt* — the adversary may submit any $x \in \Omega_{\text{change}}$ — it prevents the attempt from *counting*. And symmetrically, the fence cannot be removed silently: relaxing B is itself a policy change, hence itself an admitted change with its own receipt, reviewable in the chain like any other.

4 Inaccessibility of Invalid Change Classes

We now prove that eight classes of adversarial mutation are inaccessible to acceptance. The phrasing is deliberate: *inaccessible to acceptance* does not mean the attempt cannot be made — the threat model (Section 3.2) grants the adversary arbitrary submissions and even raw infrastructure access. It means the attempt cannot become *accepted enterprise state* without violating one of the named assumptions. For each class we give the attack as it occurs in practice, the proof, and its instantiation in the running P2P example.

Proposition 1 (Unreceipted Raw Mutation). *A direct state mutation that does not produce a valid receipt is inaccessible to acceptance.*

Proof. Consider a raw write — a SQL `UPDATE`, a direct object-store `PUT`, an edited configuration file. Submitted as a candidate x, it carries no receipt satisfying Definition 2: no canonical OCEL logs are embedded (substance fails), no challenge nonce was minted before execution (challenge binding fails), so $\text{VerifyReceipt}(R) = 0$ and conjunct C2 fails. By Definition 3, $\text{Accept}(x) = 0$; by Corollary 1, x is refused or residualized. The mutated row may physically exist in storage, but by Lemma 1 authoritative state is the receipt-chain derivation, against which the row is detectable corruption, not state.

P2P instance. The payments employee runs the raw statement `UPDATE invoice SET status='paid'` on row 4711. The row changes; the chain does not. Any consumer that derives state per Lemma 1 — the payment execution service, the auditor, the reconciliation job — sees invoice #4711 as $\mathsf{submitted}$, flags the divergent row, and the write itself becomes forensic evidence against its author.

Proposition 2 (Process-State Skip). *A change attempting a transition not admitted from the current process state is inaccessible to acceptance.*

Proof. Let the declared law admit Pay only from $\mathsf{approved}$, and let the current state be $s = \mathsf{submitted}$. For the candidate carrying $e = \mathsf{Pay}$, conjunct C6 evaluates $B(\mathsf{submitted}, \mathsf{Pay}) = 0$, so $\text{Accept}(x) = 0$ and, by Definition 5, $\mu(s, e) = s$. No combination of valid credentials, valid signature, and fresh nonce compensates: the conjuncts are independent, and C6 fails regardless of the others.

P2P instance. The classic fraud — pay an invoice that was never approved — requires the $\mathsf{submitted} \to \mathsf{paid}$ jump. Under the framework the jump is not a policy violation to be detected later; it is an inadmissible transition that never acquires state authority, the direct analogue of an unenabled transition in a workflow net [**?**].

Proposition 3 (Object Substitution). *A change that binds an event to different objects than those the authority signed over is inaccessible to acceptance.*

Proof. The signature is computed over $h(a\|e\|O\|s\|R_{\text{prior}}\|p\|v\|t\|n)$, where O is the canonical OCEL 2.0 event-to-object relation. Substituting $O' \neq O$ changes the message, so $\text{VerifySig}(\sigma, h(\cdots O' \cdots)) = 0$ (C1 fails). Re-signing over O' requires the authority's secret key, i.e., an EUF-CMA forgery against ML-DSA — a named channel, not a free move. Independently, C7 checks O' against the object lifecycles in the chain: an approval relating an invoice whose chain shows no submission fails ObjectsValid.

P2P instance. The approver reviews and approves invoice #4711; the attacker redirects the approval to invoice #4712 (their own). In a foreign-key world this is a one-column edit. Here the relation $\{\mathsf{Inv\#4711}, \mathsf{PO\#982}\}$ is inside the signed message, so the redirect invalidates the approval it tries to steal.

Proposition 4 (Backdated Change). *A change claiming an earlier effective time than its true chain position is inaccessible to acceptance as valid history.*

Proof. Time in the framework is process evidence, not a data field: the position of R_t in the chain is fixed by $h(R_{t-1})$ (Definition 2, ii). Inserting a change at past position $k < t$ requires either a second preimage of $h(R_{k-1})$ (BLAKE3 assumption) or re-signing every downstream receipt $R_k, \ldots, R_t$ (signature assumption, for every affected authority). Merely editing the timestamp field t' of an existing change alters the signed message and fails C1.

P2P instance. "This approval happened before the auditor's cutoff date." The claim requires the approval receipt to sit before the cutoff receipt in the chain. It does not, and no field edit can move it.

Proposition 5 (Replay). *A previously accepted change cannot be accepted a second time.*

Proof. The replayed candidate carries its original nonce n, already consumed, so $\text{Fresh}(n) = 0$ (C5 fails). Minting a fresh nonce does not help: n is inside the signed message, so the re-nonced candidate fails C1 without the authority key. Moreover σ binds R_{prior}; once the chain has advanced, the cited prior receipt no longer matches the chain head and C2's chaining condition fails as well. Replay is thus triply blocked — by freshness, by signature binding, and by process position.

P2P instance. A captured, perfectly valid Pay message for invoice #4711 is resubmitted to pay it twice. The duplicate-payment fraud that reconciliation teams chase quarterly is structurally inadmissible.

Proposition 6 (Schema-Valid Fraud). *A change that is syntactically and schema-valid but process-invalid is inaccessible to acceptance.*

Proof. Schema validity establishes only that the candidate parses into Ω_{change} — it is the precondition for evaluation, not a conjunct of Accept. A well-formed payload still independently requires C1 (authority), C5 (freshness), C6 (process position), and C7 (object consistency). The genre of attack in which correctly shaped data is injected through a permissive write path is exactly the conjunction "parses $\wedge$ $B(s, e) = 0$," and fails C6.

P2P instance. `{"status": "approved"}` is a perfectly valid JSON document, accepted by every schema validator. The question the framework asks is not whether the document is well-formed but whether an Approve event is admissible from the invoice's current state, emitted by an actor whose policy grant covers it, with a verifier-sealed receipt. Shape was never the issue.

Proposition 7 (Rogue Administrator Mutation). *An administrator cannot alter enterprise state except by emitting admitted changes.*

Proof. Administrative privilege is represented in exactly one place: the policy component C3, where PolicyAllowed(a, e, p) may grant actor a privileged event types. Privilege therefore enlarges the set of events an admin may *propose*; it does not waive C1, C2, C5, C6, or C7. A direct infrastructure write by an admin is Proposition 1; an in-band admin change skipping process states is Proposition 2. The admin's emergency-override path, if the enterprise wants one, must be a declared event type with its own boundary entries — making the override itself receipted, attributed, and minable.

P2P instance. The administrator "unblocks" a stalled invoice by forcing it to approved. Under the framework the only lawful version of this act is an explicit AdminOverride event — admitted from defined states, four-eyes if policy says so, and permanently visible in the chain. The governance shift is from *admin as superuser* to *admin as privileged process participant.*

Proposition 8 (AI-Agent Hallucinated Action). *An AI-agent action that does not carry the admitted-change witness is inaccessible to acceptance.*

Proof. An agent action is a candidate change x_{agent} with the agent (or its mandating principal) as actor a. Nothing in Definition 3 is relaxed for automation: the action requires a signature under a key whose policy grant covers e (C1, C3), a verifier-sealed receipt (C2), freshness (C5), and process admissibility (C6, C7). A hallucinated action — one outside the agent's mandate or the process law — fails at least one conjunct and is residualized. The residual stream is itself valuable: it is a labelled log of what the agent *attempted*, which is precisely the observability that agent-governance frameworks call for [**?**,**?**].

P2P instance. An agent tasked with "cleaning up stale invoices" attempts to cancel forty active ones. Each cancellation is a candidate Cancel event; for the active invoices, $B(\mathsf{approved}, \mathsf{Cancel}) = 0$ under the declared law, so the batch residualizes rather than executes. The agent's operator receives forty receipts of refusal — a far better failure mode than forty restored backups. This converts agentic systems from open-ended mutation engines into receipt-bearing process participants, the governance posture argued for in [**?**,**?**].

Table 1 summarises the failing conjunct for each class. Two structural observations follow from the table. First, no class depends on detecting *intent*: every block is a mechanical predicate failure, which is why the guarantees survive insiders and automation. Second, the classes fail on *different* conjuncts — the predicate is not one wall but seven, and the attack-surface analysis of the next section is the study of what it costs to breach each one.

Table 1. Attack classes and the acceptance conjunct that blocks each.

Attack class	Blocking conjunct	Prop.
Raw database/API write	C2: VerifyReceipt$(R) = 0$	1
Process-state skip	C6: $B(s, e) = 0$	2
Object substitution	C1 (σ binds O); C7 lifecycle check	3
Backdating	C2 chain position; C1 (σ binds t)	4
Replay	C5: Fresh$(n) = 0$; C1; C2	5
Schema-valid fraud	C6: $B(s, e) = 0$	6
Rogue admin direct edit	C2 or C6 (privilege only widens C3)	7
AI-agent hallucinated action	C3, C6, or C2 (no relaxation for automation)	8

5 Attack Surface Decomposition

The propositions of Section 4 hold *conditionally* on the assumption ledger of the threat model. This section makes the ledger quantitative: we bound the adversary's success probability by a union over seven named channels, characterise each channel's type and mitigation, and discuss the residual risk that no formal framework removes.

Theorem 2 (Attack-Surface Decomposition). *The probability that any invalid change is accepted satisfies*

$$\begin{aligned}\Pr[\exists\, x \notin A_{\text{enterprise}} : \text{Accept}(x) = 1] \;\leq\; & \Pr[\text{signature forgery}] \\ & + \Pr[\text{receipt forgery}] \\ & + \Pr[\text{boundary-map gap}] \\ & + \Pr[\text{policy-content gap}] \\ & + \Pr[\text{validator compromise}] \\ & + \Pr[\text{object-model gap}] \\ & + \Pr[\text{gate implementation defect}].\end{aligned}$$

Proof. An accepted invalid change requires at least one conjunct C1–C7 of Definition 3 to evaluate incorrectly — either the predicate's mathematical content is broken (its underlying hardness assumption fails) or its evaluation is wrong (the component computing it is compromised or defective). Enumerating the conjuncts: C1 fails only under signature forgery; C2 under receipt forgery (hash preimage/collision or substance-check bypass); C6 under an incomplete boundary map; C3 under incorrect policy content; C4's guarantee is void under validator compromise; C7 under an imprecise object model; and any conjunct's evaluation under a gate implementation defect. The union bound over these events gives the inequality.

5.1 Channel Characterisation

Table 2 classifies the channels; we discuss each.

Table 2. The seven break channels: type, bounding factor, and primary mitigation.

Channel	Type	Bounded by	Primary mitigation
Signature forgery	cryptographic	EUF-CMA of ML-DSA	FIPS 204/205 primitives; crypto-agility
Receipt forgery	cryptographic	BLAKE3 (2nd-)preimage	substance axiom; canonical hashing
Boundary-map gap	modelling	process-model fitness	mining residuals; reachability analysis
Policy-content gap	operational	policy review quality	policy changes are admitted changes
Validator compromise	operational	key custody	HSM keys; version revocation (C4)
Object-model gap	modelling	object-model precision	OCEL typing; lifecycle checks (C7)
Implementation defect	engineering	verification depth	proof-gate tests; determinism; refusal taxonomy

Signature forgery (cryptographic). The adversary produces a valid signature without the authority key. Under EUF-CMA security of ML-DSA (Module-LWE hardness) this probability is negligible in the security parameter [**?**,**?**]; SLH-DSA provides the hash-based fallback should lattice cryptanalysis advance [**?**]. Because the channel's assumptions can change over decades-long receipt lifetimes, algorithm identifiers are versioned receipt fields and the deployment posture follows the crypto-agility maturity model [**?**]: migration re-signs the chain head, never rewrites history.

Receipt forgery (cryptographic). The adversary constructs R with $\mathrm{VerifyReceipt}(R) = 1$ for a transition that did not occur. The hash route costs a BLAKE3 second preimage ($\geq 2^{128}$ work). The cheaper historical route — submitting hashes without substance, or a time-shifted clone of a golden fixture — is closed by Definition 2: the refusal states `PathHashOnlyReceipt`, `FixtureMutationDetected`, `SelfCertifiedAlignment`, and the challenge-nonce family (Section 6.4) are precisely the engineering instantiations of this channel's defence.

Boundary-map gap (modelling). A transition that *should* be inadmissible is absent from the declared model, so $B(s, e) = 1$ vacuously. This is the channel through which "lawful-but-wrong" changes pass, and it is not cryptographic — it is bounded by the fitness and precision of the declared process model in exactly the sense of the conformance literature [**?**]. The mitigation is the residual feedback loop of Corollary 1 combined with periodic discovery over the admitted chain (Lemma 1); formal reachability analysis of the boundary map against the declared workflow net [**?**] is future work (Section 8).

Policy-content gap (operational). The policy version is current, correctly bound, and wrong: it grants an authority it should not. Version binding (C3, with p inside σ) eliminates downgrade and substitution attacks, but no mechanism inside the framework validates policy *intent*. The structural mitigation is reflexivity — a policy change is itself an admitted change, with its own actor, receipt, and chain position — which makes policy drift minable and attributable, in line with change-management traceability requirements [**?**].

Validator compromise (operational). The component that evaluates B and seals alignments is subverted; it can then admit anything until detected. Containment, not prevention, is the realistic posture: validator keys live in HSMs, the validator version v is bound into every signature (C4), and revoking a compromised v retroactively identifies every change it admitted — the blast radius is enumerable from the chain itself. Maturity frameworks for ML-bearing validators apply here [**?**].

Object-model gap (modelling). The OCEL typing is too coarse to distinguish semantically different objects — e.g., the model checks that *an* invoice is referenced but not that it is *the* invoice whose lifecycle the chain records. C7's lifecycle consistency check closes the common cases; the channel is bounded by the precision of the object model, and richer semantic typing of the kind explored in knowledge-graph representations of process traces [**?**,**?**] narrows it further.

Gate implementation defect (engineering). The gate code returns $\mathrm{Accept}(x) = 1$ erroneously. This channel is bounded by verification depth: the reference implementation addresses it with proof-gate tests executed on every commit, bit-exact determinism (which makes gate behaviour reproducible and therefore testable), and the closed refusal taxonomy, which turns "did we handle this case?" into an enumerable question (Section 6).

5.2 What the Decomposition Buys

Three properties distinguish this claim structure from conventional security assertions.

Falsifiability. Each channel names the experiment that would break it. A purported counterexample — an accepted invalid change — must exhibit which conjunct evaluated incorrectly and why, which converts security disputes into attributable engineering claims.

Heterogeneous assurance. The channels have different epistemic statuses: two rest on cryptographic hardness (decades of public cryptanalysis), two on modelling fidelity (measurable via conformance metrics), two on operational custody (auditable via maturity models [**?**]), one on software correctness (testable). An assessor can price each independently — which is precisely what enterprise risk frameworks require [**?**].

Honest residue. The decomposition states what is *not* guaranteed. The boundary-map and policy-content gaps are irreducibly human: the framework

guarantees that the declared law is enforced exactly, not that the declared law is right. We regard this as a feature — it locates the unavoidable judgement where it belongs, in the reviewed, receipted, attributable evolution of the declared model, rather than diffusing it through ten thousand unaudited write paths.

The resulting claim, stated in full: *unauthorized change cannot be accepted as valid enterprise state unless the adversary breaks a named cryptographic assumption, exploits a gap in the declared process or object model, compromises validator custody, or finds a defect in the gate — and each of these events is independently detectable, attributable, and bounded.* That is a weaker sentence than "unhackable," and a much stronger one than the industry baseline.

6 Reference Implementation: wasm4pm

We instantiate the framework in *wasm4pm* [**?**], an open-source process mining platform. This section maps each formal component to its concrete realisation and reports the evaluation evidence: determinism guarantees, the proof-gate suite, binary footprints, and the operational verification protocol.

6.1 Architecture

wasm4pm comprises two layers. The **core** is a Rust workspace compiled to WebAssembly via `wasm-bindgen`, registering 60 process mining algorithms in a kernel registry: discovery (directly-follows graphs, Heuristic Miner [**?**], Inductive Miner [**?**], POWL 2.0 [**?**]), conformance checking (token replay [**?**], alignments), object-centric analysis (OCEL 2.0 ingestion, OC-DFG [**?**], OCPQ querying [**?**]), and supporting ML primitives. The **orchestration layer** is a TypeScript monorepo of eleven packages providing the `wpm` CLI, Zod-validated configuration, an execution planner with four deployment profiles, OpenTelemetry observability, and a Supabase-backed receipt store with an offline sync queue (five-attempt cap, dead-letter table) for intermittently connected environments.

The WASM compilation target is itself a security decision: the admission gate runs in a sandboxed, capability-free module with no ambient filesystem or network authority, embeddable in browsers, server runtimes, and edge platforms — so the verifier executes *inside the data owner's boundary*, and only hashes and receipts leave it.

6.2 Determinism as a Receipt Precondition

Lemma 1 requires that any holder of a receipt can recompute its hashes. This is only possible if the underlying computation is reproducible: a non-deterministic analysis yields different `output_hash` values on different machines, making every receipt unverifiable. wasm4pm therefore enforces bit-exact determinism as a CI merge gate — identical input must produce identical output bytes across platforms. Concretely: all randomness is seeded (stochastic algorithms currently use a fixed seed), all hash-map iteration is order-sorted before serialisation, and

Table 3. TrueX envelope fields and their formal counterparts.

Envelope field	Formal component	Content
`session_id`	a (actor/session)	originating session identity
`ocel2_batch_hash`	$h(O)$	BLAKE3 over canonical OCEL 2.0 batch
`receipt_hash`	$h(R)$	BLAKE3 over the receipt body
`admission_status`	Accept(x) outcome	`ReceiptAdmitted` iff all conjuncts hold
`equivalence_class`	s (state class)	process-state equivalence class
`expected_path_hash`	declared model path	hash of the admissible path in the model
`truex_profile`	v (validator version)	canonicalisation/verifier profile
`trace_id`, `span_id`	observability	OpenTelemetry correlation

floating-point operations avoid platform-dependent intrinsics. Canonicalisation of OCEL 2.0 documents follows a JCS-style profile (UTF-16 lexicographic key ordering, deterministic array ordering) so that hash computation is byte-stable regardless of producer formatting — the implementation of Definition 2(i).

6.3 TrueX Envelopes as Concrete Admitted Changes

The *TrueX envelope* realises a candidate change $x \in \Omega_{\text{change}}$ on the wire. Table 3 maps envelope fields to the formal components of Definition 1.

Two architectural rules protect the envelope's integrity. First, `admission_status` is *verifier-written*: the producer cannot set it, and a producer-supplied alignment claim triggers the `SelfCertifiedAlignment` refusal — the implementation of Definition 2(iii). Second, ingestion is idempotent on `receipt_hash`: replayed envelopes are detected at the persistence boundary, complementing the nonce check of C5.

6.4 The Boundary Predicate as a Refusal Taxonomy

The admission-time checks of Definitions 2 and 3 are implemented as thirteen non-overlapping refusal states, each a DENY-level verdict with a structured payload (code, severity, JSON path, message):

Ingress validation: `ObservedOCELMissing`, `ExpectedOCELMissing`, `PathHashOnlyReceipt` — the substance axiom; a receipt without recomputable logs is refused at the door.

Adversarial detection: `ObservedTraceMutationWithoutBoundary`, `FixtureMutationDetected` — structural-isomorphism and temporal-offset analysis catches near-clones of golden fixtures whose timestamps and identifiers were shifted.

Envelope verification: `RuntimeObserverMissing` — receipts claiming live execution must carry the runner envelope (runner ID, device, build, session nonces, timestamps).

Cryptographic binding: `ChallengeNonceMissing`, `ChallengeNonceMismatch`, `ObservedTraceNotChallengeBound` — the pre-minted 128-bit nonce must appear in the challenge manifest, the raw runner output, *and* the observed OCEL events (Definition 2(iv)).

Evidence binding: `BoundaryEvidenceMissing`, `SummaryOnlyReceipt` — raw executor evidence (exit codes, stdout/stderr hashes) must be bound, and observed logs must contain granular provenance events, not domain-level summaries.

Authority & hierarchy: `SelfCertifiedAlignment`, `ProofClassOverclaimed` — alignment is verifier-sealed, and no receipt may claim a proof class whose conditions it does not meet.

The taxonomy's closure property matters for the implementation-defect channel of Theorem 2: because refusals are enumerable, "is every failure mode handled?" becomes a finite checklist, each entry with dedicated negative tests that inject the corresponding forged receipt and assert refusal.

6.5 Evaluation

Proof gates. Every commit to the repository runs a proof-gate suite of 16 receipt-backed tests across seven proof files (receipt construction and hashing, release admissibility, WASM module surface, OCEL end-to-end validation, ML primitives, statistical process control, and reinforcement-learning components). The gate is enforced in the pre-push hook and CI with a hard time budget (the current suite completes in under 1.5 s), so admission-relevant regressions cannot land silently. In the terminology of Section 2.2, the suite maintains ChicagoProof status for the core pipeline: at least one test exercises the real WASM boundary end-to-end, and mocking that boundary is prohibited by policy and enforced by review tooling.

Determinism. The merge gate re-executes representative analyses and compares output bytes; a single differing byte fails the build. This has held across the browser and Node.js targets of the WASM artifact.

Footprint. The full browser-profile WASM binary is ≈ 3.5 MB (with a CI size-regression gate at +5%); size-reduced profiles (≈ 0.5–2 MB) target mobile, IoT, edge, and fog deployments. The practical consequence for the framework is deployment locality: the admission gate is small enough to run at the point of change — in the browser form, at the API gateway, on the shop-floor gateway — rather than as a remote service the data must travel to.

Operational verification. The `wpm doctor` protocol verifies the admission pipeline itself and reports a three-state progression: `prepublish_only` (credentials absent or store unreachable), `configured` (reachable, migrations applied, service role present), and `live_verified`. The last is earned by a live probe that exercises every leg of the evidence path — upsert a probe receipt, read it back, write a

dead-letter entry, ingest a TrueX envelope through the edge function — and then emits a BLAKE3-hashed *runtime receipt* recording the probe results. The status is thus itself an instance of the framework: the claim "the admission pipeline works" is only accepted when accompanied by a verifiable receipt, closing the self-reference loop.

Implementation coverage. The current deployment closes all seven conjuncts of Accept(x). Conjunct C1 is realised by `ed25519-dalek` signing over the canonical BLAKE3 hash of $(a\|e\|O\|s\|R_{\text{prior}}\|p\|v\|t\|n)$; the `sig_algorithm` field carries the algorithm identifier for crypto-agility migration (Section 5.1), with ML-DSA-65 as the declared forward target staged behind the same interface. Conjunct C5 is realised by a persistent consumed-nonce ledger (`.wasm4pm/nonces/consumed.jsonl`), checked at admission and appended on acceptance; re-submission of any consumed nonce is refused with `NonceConsumed` before the chain checks proceed. C2 chain validation is enforced by `verify_receipt_chain`, which walks the receipts directory, follows hash links from the genesis entry, and verifies each $h(R_{t-1})$ against a fresh BLAKE3 of the prior receipt body. The proof-gate suite includes one negative test per conjunct (C1–C7), each injecting a candidate that fails exactly the target conjunct and asserting the correct `failing_conjunct` field. OpenTelemetry CLI-side and WASM-side spans are emitted in the same trace context; distributed-trace correlation across the JS–WASM boundary is on the roadmap. Stochastic algorithm seeds are fixed for the determinism merge gate; a seed-audit subcommand is planned.

7 Related Work

We position the framework against eight adjacent threads.

Process-aware information systems and workflow theory. Workflow nets and their soundness theory [?,?] established that enterprise processes admit formal transition semantics, and the workflow patterns programme [?] catalogued the control-flow vocabulary a declared model may use. PAIS execute declared models; what they historically did not do is deny state authority to changes arriving *outside* the engine — the database write behind the workflow system's back remains effective. Our framework closes exactly that gap: Definition 5 withholds state authority from any transition the declared law does not admit, regardless of arrival path.

Process mining and conformance checking. Discovery and conformance [?,?,?] relate completed logs to models via token replay [?] and alignments [?,?], scored on the four quality dimensions [?]. We inherit this entire toolbox but invert its temporal position: conformance becomes an admission gate rather than a measurement, and the boundary-map channel of Theorem 2 is bounded by precisely the fitness/precision quantities this literature knows how to measure.

Compliance-aware predictive monitoring [?] moves in the same prospective direction — predicting violations before completion — but remains advisory; our gate is constitutive.

Object-centric process mining. OCEL [?], its 2.0 standard [?,?], object-centric Petri nets [?], OC-DFGs [?], and object-centric constraint languages [?,?] supply the multi-object event semantics without which Proposition 3 cannot even be stated. Van der Aalst's "no AI without PI" argument [?] holds that object-centric process intelligence must ground enterprise AI; our Proposition 8 is the enforcement-side complement. Semantic enrichments of event data — knowledge-graph trace querying [?] and OCED/SPARQL integration [?] — sharpen the object-model channel of Section 5.1.

Blockchain-anchored process execution. Caterpillar [?] and successors execute BPMN on Ethereum, obtaining tamper-evidence from consensus. The trilemma analysis [?] quantifies what that costs in throughput and operational complexity. Our design point differs: integrity comes from local hash chains plus signatures, consensus is unnecessary because the validator — not a miner quorum — is the admission authority, and the evidence stays inside the data owner's boundary. The trade is explicit: we give up Byzantine fault tolerance among mutually distrusting writers (which single-enterprise governance rarely needs) and keep millisecond admission latency and data locality.

Access control. ABAC/XACML [?] answers *who may attempt what*; conjunct C3 embeds exactly such a policy decision. The remaining six conjuncts have no access-control analogue: an authorised actor's change still fails on receipt substance, freshness, process position, or object consistency. Access control and change admissibility compose; neither subsumes the other (Proposition 7 is the sharpest separation — privilege widens C3 and nothing else).

Anomaly and fraud detection. Statistical detectors over event streams [?,?] are probabilistic and post-hoc: they score suspicion. Our guarantees are categorical at the gate, but the approaches compose rather than compete — detectors running over the residual stream (Corollary 1) get a labelled, complete, tamper-evident input, and what they find feeds the boundary-map review loop.

Provenance and evidence engineering. W3C-style provenance over semantic-web data [?], type-safe process-evidence construction [?], semantic audit models for cloud engines [?], and structured threat-intelligence schemas [?] all pursue machine-checkable records of what happened. The receipt chain of Section 2.2 is a member of this family with two distinguishing commitments: the substance axiom (no hash-only records) and pre-admission positioning (the record gates the change instead of describing it).

Governance and maturity frameworks. Security and AI maturity models [?,?,?], crypto-agility assessment [?], PQC migration mandates [?], change-management traceability [?], and enterprise-architecture risk integration [?] describe the organisational scaffolding into which a deployment of this framework must fit. The attack-surface decomposition (Table 2) is designed to be legible to exactly these instruments: each channel maps to an assessable control domain, and AI-agent governance proposals [?,?] obtain in Proposition 8 a mechanical enforcement substrate for mandates that are otherwise procedural.

8 Conclusion

We have presented the Enterprise Change Admissibility Framework: a formal extension of process mining's admitted-event semantics into a governance law for all enterprise state change. Its content is captured by three statements of increasing concreteness. Mathematically, acceptance is membership — $\mathrm{Accept}(x) \Rightarrow x \in A_{\text{enterprise}}$ — and there is no second path into enterprise state (Theorem 1, Lemma 1). Adversarially, eight recurring classes of enterprise mutation — raw writes, state skips, object substitution, backdating, replay, schema-valid fraud, rogue administrator edits, and AI-agent hallucinated actions — are inaccessible to acceptance except through seven named, typed, independently bounded break channels (Sections 4, 5). Practically, the framework is implementable today: wasm4pm realises it with deterministic WASM execution, BLAKE3 receipt chains over canonical OCEL 2.0, a thirteen-state refusal taxonomy, and post-quantum-ready signing (Section 6).

The deeper claim is a temporal one. Process mining has spent two decades perfecting the reconstruction of what happened; the machinery it built — event semantics, object-centric logs, conformance measures, boundary predicates — turns out to be exactly the machinery needed to govern what is *allowed to happen*. Moving that machinery from after the fact to before acceptance changes the question an enterprise system answers, from "can this input be parsed and authorized?" to "can this change prove it is a lawful step of a declared process?" Every result in this paper is downstream of that single repositioning.

Limitations and future work. Four items delimit the present contribution. *(1) Boundary completeness.* The refusal taxonomy is grounded in observed attack patterns; a completeness proof against the threat model — e.g., via reachability analysis of the boundary map over the declared workflow net [?] — would convert the boundary-map channel from empirically narrowed to formally bounded. *(2) Mechanised proofs.* The propositions of Section 4 are short enough to formalise; a mechanised development (e.g., in a proof assistant, following the type-safe evidence programme of [?]) would also sharpen the gate-defect channel. *(3) Implementation gaps.* Signature integration, CLI–WASM trace correlation, and configurable audited seeds are staged work in the reference implementation (Section 6.5). *(4) Institutional acceptance.* Receipts carry evidential weight in regulated contexts only once audit institutions accept them as compliance artifacts; the framework

supplies the verifiable object, but the endorsement is organisational, and the maturity-model literature [**?**,**?**] suggests the assessment vocabulary through which it will arrive.

Closing. Every enterprise security question is eventually a change question: who changed the role, the invoice, the policy, the model, the permission. Systems that answer by reconstruction will always answer too late. The framework presented here answers by construction — no change enters enterprise state except through an admitted, signed, object-linked, process-positioned, receipt-bearing event — and it makes the cost of every exception explicit. Attackers, insiders, and agents may still submit anything they wish. What they can no longer do, short of breaking a named assumption, is make it count.

References

1. Towards dependable change management and traceability for global software development. Conference paper (2016), change governance and distributed traceability requirements; motivates the admitted-change witness architecture.
2. An integrated conceptual model for information system security risk management and enterprise architecture management based on TOGAF, ArchiMate, IAF and DoDAF. Conference paper (2017), enterprise architecture and security integration. Provides institutional context for the enterprise change admissibility framework deployment.
3. Towards a maturity model for crypto-agility assessment. Preprint (2022), framework for assessing an organisation's readiness to migrate cryptographic primitives; directly relevant to the PQC channel of the attack decomposition.
4. Object-centric event log standard v2.0 (2023), https://www.ocel-standard.org/, oCEL 2.0 community reference. Resources paper at ICPM 2023 (Koren et al., CEUR-WS Vol. 3648, paper_7195). Classified consumer_contract_only: cited for format compliance, not algorithm grounding.
5. Evolving AI risk management: A maturity model based on the NIST AI risk management framework. Preprint (2024), nIST AI RMF applied as a maturity model; relevant to AI-agent governance (Proposition 8).
6. On the ETHOS of AI agents: An ethical technology and holistic oversight system. Preprint (2024), holistic oversight framework for AI agents; motivates receipt-bearing participation requirement for agents.
7. Process trace querying using knowledge graphs and Notation3. Preprint (2024), represents event logs as knowledge graphs; SPARQL/N3 querying of process traces. Relevant to object-model precision (Section 5).
8. Quantifying the blockchain trilemma: A comparative analysis of algorand, Ethereum 2.0, and beyond. Preprint (2024), analyses decentralisation-security-scalability tradeoffs. Motivates the local receipt chain approach over distributed ledger for enterprise admission.
9. SoK: Identifying limitations and bridging gaps of cybersecurity capability maturity models. Preprint (2024), systematic analysis of security maturity models; provides vocabulary for the operational tier of the attack decomposition (Section 5).
10. Type-safe process-evidence engineering. Preprint (2024), type-safe proofs of execution; cryptographic receipt chains; formal evidence theory for admitted process transitions.

11. Enhancing data integrity through provenance tracking in semantic Web frameworks. Preprint (2025), rDF-based provenance tracking and cryptographic integrity chains; semantic web complement to OCEL 2.0 receipt model.
12. van der Aalst, W.M.P.: Verification of workflow nets. In: Application and Theory of Petri Nets 1997 (ICATPN 1997). Lecture Notes in Computer Science, vol. 1248, pp. 407–426. Springer (1997). https://doi.org/10.1007/3-540-63139-9_48, peer-reviewed. Springer link.springer.com/chapter/10.1007/3-540-63139-9_48 confirmed. Original WF-net soundness paper. Definitions: single source/sink, option to complete, dead-transition freedom, boundedness.
13. van der Aalst, W.M.P.: Process Mining: Discovery, Conformance and Enhancement of Business Processes. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19345-3, first edition PM textbook. Defines play-in, play-out, and replay as conceptual triad; establishes generalization and simplicity as quality dimensions. Grounds playout algorithm concept.
14. van der Aalst, W.M.P.: Process Mining: Data Science in Action. Springer, Berlin, Heidelberg, 2nd edn. (2016). https://doi.org/10.1007/978-3-662-49851-4, canonical textbook reference for DFG (Ch. 3), WF-net soundness, generalization metrics (Sec. 9.3), process trees (Ch. 6), and quality dimensions. Cited broadly across all algorithm families.
15. van der Aalst, W.M.P.: Conformance Checking: Relating Processes and Models. Springer (2018), supports conformance checking algorithm family. Classified supports_family in citation inventory C22.
16. van der Aalst, W.M.P.: No AI without PI! object-centric process mining as the enabler for generative, predictive, and prescriptive artificial intelligence. Technical Report (2024), argues that object-centric process mining is a prerequisite for trustworthy AI in enterprise contexts. Foundational motivation for applying OCEL 2.0 to AI agent governance.
17. van der Aalst, W.M.P., Berti, A.: Discovering object-centric petri nets. Fundamenta Informaticae **175**(1–4), 1–40 (2020). https://doi.org/10.3233/FI-2020-1946, peer-reviewed journal. DBLP confirmed at dblp.org/db/journals/fuin/fuin175.html. IOS Press ebooks confirmed. arXiv:2010.02047. First formally complete process model for object-centric event data.
18. Adriansyah, A.: Aligning Observed and Modeled Behavior. PhD thesis, Technische Universiteit Eindhoven (2014). https://doi.org/10.6100/IR770080, 252 pages. ISBN: 978-90-386-3574-3. SIKS Dissertation Series. TU/e Research Portal confirmed at research.tue.nl/en/publications/aligning-observed-and-modeled-behavior/ and full text at research.tue.nl/files/4032919/770080.pdf. Full formal treatment of alignment-based conformance.
19. Adriansyah, A., van Dongen, B.F., van der Aalst, W.M.P.: Conformance checking using cost-based fitness analysis. In: Proceedings of the 15th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2011). pp. 55–64. IEEE (2011). https://doi.org/10.1109/EDOC.2011.12, peer-reviewed. IEEE Xplore confirmed at ieeexplore.ieee.org/document/6037560/. DBLP confirmed at dblp.org/pid/90/7976.html. First peer-reviewed paper introducing cost-based alignment for conformance; A* over product automaton of log trace and Petri net.
20. Bernstein, D.J., Lange, T.: Post-quantum cryptography. Nature **549**(7671), 188–194 (2017). https://doi.org/10.1038/nature23461, survey of post-quantum cryptography landscape. Motivates migration from ECDSA/RSA for long-lived signatures.
21. Berti, A., van der Aalst, W.M.P.: OC-PM: Analyzing object-centric event logs and process models. International Journal on Software Tools for Technology Transfer

25, 1–17 (2023). https://doi.org/10.1007/s10009-022-00668-w, peer-reviewed journal. Springer confirmed at link.springer.com/article/10.1007/s10009-022-00668-w. arXiv:2209.09725. Formal multigraph definition of OC-DFG.
22. Berti, A., Koren, I., Adams, J.N., Park, G., et al.: OCEL (Object-Centric Event Log) 2.0 specification. arXiv:2403.01975v1 [cs.DB] (2024), https://arxiv.org/abs/2403.01975, oCEL 2.0 specification. 13 named authors. Adds O2O relationships, dynamic object attributes, qualified arc types; three exchange formats: SQLite, XML, JSON. arXiv preprint of spec document.
23. Berti, A., van Zelst, S., Verbeek, E.: PM4Py: A Process Mining Library for Python. Technical Report, RWTH Aachen (2023), standard Python process mining library. Provides the baseline algorithm implementations against which wasm4pm's deterministic WASM core is benchmarked.
24. Bose, R.P.J.C., van der Aalst, W.M.P., Žliobaitė, I., Pechenizkiy, M.: Dealing with concept drifts in process mining. IEEE Transactions on Neural Networks and Learning Systems **25**(1), 154–171 (2014). https://doi.org/10.1109/TNNLS.2013.2278313, peer-reviewed. IEEE TNNLS journal. Canonical PM concept drift paper; grounds detect_drift and compute_ewma.
25. Buijs, J.C.A.M., van Dongen, B.F., van der Aalst, W.M.P.: On the role of fitness, precision, generalization and simplicity in process discovery. In: OTM 2012 Confederated International Conferences (CoopIS/DOA/ODBASE). Lecture Notes in Computer Science, vol. 7565, pp. 305–322. Springer (2012). https://doi.org/10.1007/978-3-642-33606-5_19, peer-reviewed. Springer confirmed. DBLP confirmed. First formalization of all four quality dimensions (fitness, precision, generalization, simplicity) in a unified framework.
26. Chatman, S.: wasm4pm: A deterministic WebAssembly process mining platform. GitHub repository and npm package (2026), https://github.com/seanchatmangpt/wasm4pm, 60 deterministic process mining algorithms compiled from Rust to WebAssembly. Implements the Enterprise Change Admissibility Framework via TrueX envelopes, BLAKE3 receipt chains, and 13-state boundary predicate taxonomy.
27. Chesterton, G.K.: Orthodoxy. Dodd, Mead and Company, New York (1908), original statement of the Chesterton's Fence principle (Chapter 4): do not remove a fence until you understand why it was built.
28. De Santis, F., Park, G., van der Aalst, W.M.P., Zanichelli, F.: Compliance-aware predictive process monitoring: A neuro-symbolic approach. In: Proceedings of BPM 2024 (2024), combines boundary predicates and neural prediction for pre-emptive compliance checking; directly related to the boundary predicate $B(s, e)$ model.
29. Desel, J., Esparza, J.: Free Choice Petri Nets, Cambridge Tracts in Theoretical Computer Science, vol. 40. Cambridge University Press (1995), definitive monograph on free-choice Petri nets. Admits polynomial soundness checking via Rank Theorem. Tractable target class for automated conformance and POWL translation.
30. Fernández Saura, P., Jayaram, K.R., Isahagian, V., Bernal Bernabé, J., Skarmeta, A.: On automating security policies with contemporary LLMs. In: Proceedings of IEEE Security & Privacy Workshop (2024), aI agent tool use and automated security policy enforcement; connects to the AI-agent residualization model (Proposition 8).
31. Ghahfarokhi, A.F., Park, G., Berti, A., van der Aalst, W.M.P.: OCEL: A standard for object-centric event logs. In: New Trends in Database and Information Systems — ADBIS 2021 Short Papers, Doctoral Consortium and Workshops: DOING, SIMPDA, MADEISD, MegaData, CAoNS. Communications in Computer and Information Science, vol. 1450, pp. 169–175. Springer (2021).

https://doi.org/10.1007/978-3-030-85082-1_16, peer-reviewed. Springer confirmed at link.springer.com/chapter/10.1007/978-3-030-85082-1_16. SIMPDA workshop at ADBIS 2021, Tartu, Estonia, August 24-26, 2021. OCEL 1.0 format specification. Authors: 4 (not 3 as in earlier stub).

32. Jedrzejewski, F.V., Fucci, D., Adamov, O.: MLSMM: Machine learning security maturity model. Preprint (2023), aI/ML governance and maturity assessment; informs the validator-compromise channel of Theorem 2.
33. Ko, J., Comuzzi, M.: Detecting anomalies in business process event logs using statistical leverage. Information Sciences **549**, 53–67 (2021), peer-reviewed journal. Information-theoretic edge-frequency anomaly scoring; grounds ml_anomaly algorithm.
34. Kourani, H., van der Aalst, W.M.P.: POWL 2.0: Choice graphs and frequent transitions. In: CEUR Workshop Proceedings. vol. 3783 (2024), cEUR-WS vol. 3783. Introduces FrequentTransitionNode (min_freq, max_freq API). Definition 5 (MineDG) cited as correctness oracle for POWL 2.0 discovery in wasm4pm.
35. Küsters, A., van der Aalst, W.M.P.: OC-DECLARE: Discovering object-centric declarative patterns with synchronization. In: Business Process Management — BPM 2025. Lecture Notes in Computer Science, vol. 16044, pp. 162–179. Springer (2025). https://doi.org/10.1007/978-3-032-02867-9_11, peer-reviewed. BPM 2025, Seville, Spain. First paper on automated OC-DECLARE discovery from OCEL.
36. Küsters, A., van der Aalst, W.M.P.: OCPQ: Object-centric process querying and constraints. In: RCIS (1) 2025. pp. 383–400 (2025). https://doi.org/10.1007/978-3-031-92474-3_23, peer-reviewed conference (RCIS 2025, 19th International Conference). arXiv:2506.11541. Formal Definitions 1–9 for OCPQ query tree semantics. Rust implementation at github.com/aarkue/ocpq.
37. Latif, S., Latif, H., Rahman, M.R.U.: Object-centric analysis of XES event logs: Integrating OCED modeling with SPARQL queries. Preprint (2024), semantic integration of OCEL with SPARQL; provides query layer over the object-event relations used in Proposition 3 (object substitution).
38. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs — a constructive approach. In: Application and Theory of Petri Nets and Concurrency (Petri Nets 2013). Lecture Notes in Computer Science, vol. 7927, pp. 311–329. Springer, Milan, Italy (2013). https://doi.org/10.1007/978-3-642-38697-8_17, peer-reviewed. Springer confirmed at link.springer.com/chapter/10.1007/978-3-642-38697-8_17. DBLP confirmed at dblp.org/db/conf/apn/pn2013.html and dblp.org/pid/131/1671.html. June 24-28, 2013. Inductive Miner with soundness guarantee; XOR/AND/LOOP/Sequence cut taxonomy.
39. Lopez-Pintado, O., García-Bañuelos, L., Dumas, M., Weber, I., Ponomarev, A.: Caterpillar: A business process execution engine on the Ethereum blockchain. In: Information Systems. vol. 90, p. 101470. Elsevier (2019). https://doi.org/10.1016/j.is.2019.07.009, blockchain-based process execution. Provides immutable audit trail. Contrasts with our approach: we do not require a distributed ledger.
40. National Institute of Standards and Technology: Module-lattice-based digital signature standard (ML-DSA). Tech. Rep. FIPS 204, U.S. Department of Commerce (2024), https://doi.org/10.6028/NIST.FIPS.204, based on CRYSTALS-Dilithium. Specifies ML-DSA-44, ML-DSA-65, ML-DSA-87. Primary signing algorithm recommended for enterprise receipt chains.

41. National Institute of Standards and Technology: Stateless hash-based digital signature standard (SLH-DSA). Tech. Rep. FIPS 205, U.S. Department of Commerce (2024), https://doi.org/10.6028/NIST.FIPS.205, based on SPHINCS+. Hash-based; conservative fallback when lattice assumptions are uncertain.
42. OASIS: eXtensible access control markup language (XACML) version 3.0. Tech. rep., OASIS Standard (2013), https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html, attribute-based access control standard. Governs who may initiate change; orthogonal to admissibility of the change itself.
43. OASIS Cyber Threat Intelligence (CTI) TC: STIX version 2.1. Tech. rep., OASIS Standard (2022), https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html, structured threat intelligence schema; formalises event representation with object relations; relevant to the TrueX envelope design.
44. Presman, D.: The U.S. government's transition to post-quantum cryptography. Government report (2024), describes NIST PQC migration timelines and mandates. Motivates long-lived receipt signing with ML-DSA.
45. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. Information Systems **33**(1), 64–95 (2008). https://doi.org/10.1016/j.is.2007.07.001, peer-reviewed journal. ScienceDirect confirmed at sciencedirect.com/science/article/abs/pii/S030643790700049X. DBLP confirmed at dblp.org/pid/20/5156.html. Canonical token replay paper; fitness formula fitness = 1 - (missing + consumed) / (produced + remaining) implemented verbatim in wasm4pm.
46. Russell, N., ter Hofstede, A.H.M., van der Aalst, W.M.P., Mulyar, N.: Workflow Patterns: The Definitive Guide. MIT Press (2015), canonical workflow pattern classification and control flow semantics. Grounds the boundary predicate state space in established workflow theory.
47. Sargolzaei Javan, M.: A semantic model for audit of cloud engines based on ISO/IEC TR 3445:2022. Preprint (2022), formal semantic audit model for cloud infrastructure. Provides ISO-grounded vocabulary for cloud-resident evidence chains.
48. Weijters, A.J.M.M., van der Aalst, W.M.P., Alves de Medeiros, A.K.: Process mining with the HeuristicsMiner algorithm. Tech. Rep. WP 166, BETA Working Paper Series, Eindhoven University of Technology (2006), 346+ citations per Semantic Scholar. Canonical HeuristicsMiner technical report. Dependency formula and noise-tolerance thresholds implemented directly in wasm4pm. Not a conference/journal paper; classified tech report.